

Rotary Position Embeddings.

RoPE injects *absolute* position by rotating each 2D sub-vector of the query and key by an angle proportional to its position. The magic: because rotations compose by adding angles, the attention dot product ends up depending only on the *relative* offset $n - m$. We derive that property from the rotation-composition identity, then rotate a $d=4$ query at position $m=2$ and key at $n=5$ and confirm the score collapses to a single relative rotation by 3. Every number below is computed in full — nothing summarised or deferred.

THE EQUATION

$$(R(m)q)^\top (R(n)k) = q^\top R(n-m)k \quad R(\phi) = \begin{bmatrix} \cos \phi & -\sin \phi \\ \sin \phi & \cos \phi \end{bmatrix}$$

Worked example. $d = 4$ (two frequency pairs), $\theta_0 = 1$, $\theta_1 = 10000^{-2/4} = 0.01$; query at $m = 2$, key at $n = 5$, showing the score equals the unrotated vectors rotated by the relative offset $n - m = 3$.

Topic 1 of 3 in this module · companion to the course page · v1.0

Contents

1	The claim: absolute rotations, relative scores	2
2	Frequencies: a geometric spectrum	3
3	The derivation: why only the offset survives	3
4	Worked example: $d = 4, m = 2, n = 5$	4
5	Properties / consequences that matter	5
6	Where this sits in the track	5
A	Reproducible (NumPy)	6

1 The claim: absolute rotations, relative scores

Sinusoidal and learned positional embeddings *add* a position-dependent vector to the token embedding. RoPE instead *multiplies*: it splits the d -dimensional query and key into $d/2$ pairs of coordinates and *rotates* the i -th pair by an angle $m\theta_i$, where m is the absolute token position and θ_i a fixed per-pair frequency. The governing identity is:

$$(R(m\theta_i) q^{(i)})^\top (R(n\theta_i) k^{(i)}) = q^{(i)\top} R((n-m)\theta_i) k^{(i)} \quad (1)$$

The left side encodes the two *absolute* positions m and n independently; the right side depends only on their *difference* $n - m$. Stacking all $d/2$ pairs into a block-diagonal rotation $\mathcal{R}(m) = \text{diag}(R(m\theta_0), \dots, R(m\theta_{d/2-1}))$ gives the full-vector form $(\mathcal{R}(m)q)^\top (\mathcal{R}(n)k) = q^\top \mathcal{R}(n-m)k$.

Symbol	Meaning	Value / shape
d	head dimension (even)	scalar; here 4
q, k	query / key vectors	$\in \mathbb{R}^d$
m, n	absolute positions of query / key	integers
θ_i	frequency of pair i	$\theta_i = \text{base}^{-2i/d}$
base	frequency base	10000
$R(\phi)$	2×2 rotation by angle ϕ	$\in \mathbb{R}^{2 \times 2}$
$\mathcal{R}(m)$	block-diagonal rotation	$\in \mathbb{R}^{d \times d}$

Think of each coordinate pair as the hand of a clock. Position m spins every clock forward by $m\theta_i$; a high-frequency pair (θ_i near 1) sweeps fast, a low-frequency pair ($\theta_i \rightarrow 0$) barely moves. When a query at m meets a key at n , what survives in the dot product is not where each hand points in absolute terms, but the *angle between them* — and that angle is fixed by $n - m$. So the model reads positions absolutely but compares them relatively.

2 Frequencies: a geometric spectrum

The per-pair frequencies are a geometric sequence between 1 and base^{-1} :

$$\theta_i = \text{base}^{-2i/d}, \quad i = 0, 1, \dots, \frac{d}{2} - 1.$$

For $d = 4$ there are two pairs:

$$\theta_0 = 10000^{-0/4} = 10000^0 = 1, \quad \theta_1 = 10000^{-2/4} = 10000^{-1/2} = \frac{1}{100} = 0.01.$$

Pair 0 rotates one radian per position; pair 1 rotates a hundredth of a radian per position. The fast pair resolves nearby tokens; the slow pair stays nearly fixed over long ranges, giving the representation a multi-scale “ruler” for distance.

3 The derivation: why only the offset survives

The whole construction rests on one fact about rotation matrices: rotating by a then by b is the same as rotating by $a + b$, and the transpose (inverse) reverses the angle.

3.1 The composition identity $R(a)^\top R(b) = R(b - a)$

A rotation matrix is orthogonal with $R(\phi)^\top = R(-\phi)$, and rotations add: $R(\alpha)R(\beta) = R(\alpha + \beta)$. Therefore

$$R(a)^\top R(b) = R(-a)R(b) = R(b - a).$$

Concretely, expanding with the angle-difference identities $\cos b \cos a + \sin b \sin a = \cos(b - a)$ and $\sin b \cos a - \cos b \sin a = \sin(b - a)$:

$$R(a)^\top R(b) = \begin{bmatrix} \cos a & \sin a \\ -\sin a & \cos a \end{bmatrix} \begin{bmatrix} \cos b & -\sin b \\ \sin b & \cos b \end{bmatrix} = \begin{bmatrix} \cos(b - a) & -\sin(b - a) \\ \sin(b - a) & \cos(b - a) \end{bmatrix} = R(b - a).$$

3.2 Apply to one query/key pair

Set $a = m\theta_i$ and $b = n\theta_i$. The rotated dot product for pair i is

$$(R(m\theta_i)q^{(i)})^\top (R(n\theta_i)k^{(i)}) = q^{(i)\top} R(m\theta_i)^\top R(n\theta_i) k^{(i)} = q^{(i)\top} R((n - m)\theta_i) k^{(i)}.$$

That is exactly Eq. (1). The absolute positions m and n have cancelled into the single offset $n - m$. Summing over all pairs gives the same statement for the full vectors.

The dot product is a *bilinear* form, and $R(m)^\top R(n)$ is the only place position enters. Because $R(m)^\top R(n) = R(n - m)$, the attention logit between a query at m and a key at n is a function of the gap $n - m$ alone — RoPE turns absolute encoding into relative attention *for free*, with no learned position parameters and no extra additive terms.

4 Worked example: $d = 4$, $m = 2$, $n = 5$

Take a query at position $m = 2$ and a key at position $n = 5$, so the offset is $n - m = 3$. Use

$$q = \begin{bmatrix} 1 \\ 0 \\ 1 \\ 1 \end{bmatrix}, \quad k = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}, \quad \theta_0 = 1, \theta_1 = 0.01.$$

The rotation angles are $m\theta_0 = 2$, $m\theta_1 = 0.02$ for the query and $n\theta_0 = 5$, $n\theta_1 = 0.05$ for the key (radians).

4.1 Rotate the query at $m = 2$

Pair 0 rotates by 2 rad ($\cos 2 = -0.416147$, $\sin 2 = 0.909297$); pair 1 by 0.02 rad ($\cos 0.02 = 0.999800$, $\sin 0.02 = 0.019999$):

$$\mathcal{R}(2)q = \begin{bmatrix} \cos 2 & -\sin 2 \\ \sin 2 & \cos 2 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} \mid \begin{bmatrix} \cos 0.02 & -\sin 0.02 \\ \sin 0.02 & \cos 0.02 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} -0.416147 \\ 0.909297 \\ 0.979801 \\ 1.019799 \end{bmatrix}.$$

4.2 Rotate the key at $n = 5$

Pair 0 by 5 rad ($\cos 5 = 0.283662$, $\sin 5 = -0.958924$); pair 1 by 0.05 rad ($\cos 0.05 = 0.998750$, $\sin 0.05 = 0.049979$):

$$\mathcal{R}(5)k = \begin{bmatrix} \cos 5 & -\sin 5 \\ \sin 5 & \cos 5 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} \mid \begin{bmatrix} \cos 0.05 & -\sin 0.05 \\ \sin 0.05 & \cos 0.05 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0.958924 \\ 0.283662 \\ 0.998750 \\ 0.049979 \end{bmatrix}.$$

4.3 The rotated score

$$\begin{aligned} (\mathcal{R}(2)q)^\top (\mathcal{R}(5)k) &= \underbrace{(-0.416147)(0.958924) + (0.909297)(0.283662)}_{\text{pair 0} = -0.141120} \\ &\quad + \underbrace{(0.979801)(0.998750) + (1.019799)(0.049979)}_{\text{pair 1} = 1.029546} = 0.888426. \end{aligned}$$

4.4 Confirm it equals $q^\top R(n-m)k$

Now rotate *only* k by the relative offset $n-m=3$ and dot with the *unrotated* q . Pair 0 rotates by $3\theta_0 = 3$ rad, pair 1 by $3\theta_1 = 0.03$ rad:

$$R(3) \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} -0.141120 \\ -0.989992 \end{bmatrix}, \quad R(0.03) \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0.999550 \\ 0.029996 \end{bmatrix}.$$

$$q^\top \mathcal{R}(3)k = \underbrace{(1)(-0.141120) + (0)(-0.989992)}_{-0.141120} + \underbrace{(1)(0.999550) + (1)(0.029996)}_{1.029546} = 0.888426.$$

Identical to the rotated score above, pair by pair — the absolute angles 2, 5 and 0.02, 0.05 have collapsed into the single relative rotations 3 and 0.03. The heatmap shades the two pairs' shared contributions (intensity scales with $|\cdot|$, capped at `indigo!80`):

	pair 0	pair 1	total
contribution	-0.1411	1.0295	0.8884

For reference, the *unrotated* score $q^\top k = (1)(0) + (0)(1) + (1)(1) + (1)(0) = 1$: position changes the logit, but only through the offset 3.

5 Properties / consequences that matter

- Relative by construction.** The logit between positions m, n depends only on $n-m$, without any learned relative-position table (contrast Module 1.4's disentangled attention).
- Norm-preserving.** Rotations are orthogonal, so $\|\mathcal{R}(m)q\| = \|q\|$; RoPE changes *angles*, never magnitudes, leaving the softmax scale untouched.
- No parameters, no extra adds.** RoPE is applied inside the attention as a deterministic rotation of q and k only — V is left alone, and there is nothing to train.
- Long-context extension.** A model trained to context T has the slow pairs barely past their first cycle. *Position interpolation* rescales positions $m \mapsto mT/T'$ to fit a longer T' into the trained angle range; *NTK-aware* scaling instead increases the base ($10000 \mapsto 10000 \cdot s$) so high frequencies are stretched less than low ones — both extend context with little or no retraining.

RoPE is *not* translation-invariant in the absolute angle: it is the *dot product* that is relative, not the rotated vectors themselves. If you cache rotated keys (RoPE applied *before* the KV-cache, as is standard) the cache is correct because each key carries its own absolute n ; but if you ever shift the position origin you must re-rotate. Also, the offset relation holds exactly only when the *same* θ_i schedule is used for q and k — mixing frequencies (e.g. a mis-applied interpolation) breaks $R(m)^\top R(n) = R(n-m)$ and silently corrupts long-range scores.

6 Where this sits in the track

RoPE is the position scheme of the LLaMA family and most modern decoders. It replaces the additive sinusoidal embedding of Module 0.1 and the learned relative tables of Module 1.4

with a parameter-free rotation that is both absolute (each token knows its index) and relative (attention sees only gaps). Topic 2 shows how the rotated keys are stored in the KV-cache under grouped-query attention; the long-context extensions sketched above are the entry point to extended-context training later in the track.

A Reproducible (NumPy)

Inputs: $d = 4$, $\text{base} = 10000$ so $\theta_0 = 1$, $\theta_1 = 0.01$; $q = [1, 0, 1, 1]$, $k = [0, 1, 1, 0]$; $m = 2$, $n = 5$. The snippet reproduces the rotated vectors, the score 0.888426, and the relative-form check.

```
import numpy as np
th = [1.0, 0.01] # theta_0, theta_1 (base=10000, d=4)
def R(a): c,s=np.cos(a),np.sin(a); return np.array([[c,-s],[s,c]])
def rope(v,p):
    o=v.copy()
    for i,t in enumerate(th): o[2*i:2*i+2]=R(p*t)@v[2*i:2*i+2]
    return o
q=np.array([1.,0.,1.,1.]); k=np.array([0.,1.,1.,0.]); m,n=2,5
qm, kn = rope(q,m), rope(k,n)
print(qm) # [-0.416147  0.909297  0.979801  1.019799]
print(kn) # [ 0.958924  0.283662  0.998750  0.049979]
print(qm@kn) # 0.8884255258916...
print(q @ rope(k, n-m)) # 0.8884255258916... (depends only on n-m=3)
print(q@k) # 1.0 (unrotated)
```